command: `VITE_NAME=VITE_NAME=RenderComponentExample npm run dev`.

The example of `use(promise)` is located in `use-cases/UsePromiseExample.tsx`, and it can be executed with the command: `VITE_NAME=VITE_NAME=UsePromiseExample npm run dev`.

A.4.4 Comparison between the use API and Hooks

While React Hooks provide foundational tools for state and side-effect management, the `use` API introduces a streamlined approach for asynchronous data handling and context access. Table A.10 outlines a comparison of their key differences.

Table A.10 Comparison between the use API and Hooks

Aspect	use() API	Hooks
Asynchronous	Designed for simplifying asynchronous data consumption in the render phase, integrating seamlessly with Suspense and Server Components. It handles promises directly (e.g., `const data = use(fetchData)`).	Versatile tools for managing state and side effects in functional components. Requires manual handling via `useEffect`, `useState`, and other hooks. It is called outside the render phase.
Context	A more concise way of using context. (e.g., `const theme = use(ThemeContext)`).	Traditional method using `useContext` for accessing shared state.
Conditional usage	Can be called conditionally.	Cannot be called conditionally.

A.5 React's component lifecycle